MimicCall: Bypassing System Call Filters via Kernel Function Redundancy

Songah Joo
Korea University
Seoul, Republic of Korea
syscall@korea.ac.kr

Minchan Park
Korea University
Sejong, Republic of Korea
stin@korea.ac.kr

Hyerean Jang
Korea University
Seoul, Republic of Korea
hr_jang@korea.ac.kr

Youngjoo Shin
Korea University
Seoul, Republic of Korea
syoungjoo@korea.ac.kr

***Abstract*—Modern operating systems often employ system call filtering to limit untrusted code's access to kernel resources, thereby reducing the kernel attack surface. However, existing filters operate at the system call interface level and often overlook the internal reuse of kernel functions across multiple system calls. This paper identifies and systematically analyzes a class of filter bypasses we term *MimicCalls*, wherein alternative, permitted system calls invoke the same vulnerable functions as restricted ones. We present an automated analysis framework that traces syscall-to-function mappings using ftrace, leveraging system calls' test cases to cover diverse system call behaviors, using Syzkaller. Our evaluation of a recent Linux kernel demonstrates that even semantically distinct system calls share extensive kernel logic, and that thousands of patched CVE functions are accessible via multiple call paths. We further analyze filters generated by five representative tools and show that all tools are exposed of vulnerable functions through allowed MimicCalls. Case studies on CVE-2016-4486, CVE-2016-9793, and CVE-2017-7184 validate the practical impact of our findings, demonstrating that real-world exploits can be adapted to bypass filtering. Our results reveal a fundamental limitation in system call level defenses and underscore the need for more semantically informed filtering strategies.**

***Index Terms*—Software security, System call filtering, Mimicry attack**

1. Introduction

In modern operating systems, system calls serve as the fundamental interface facilitating transitions from user space to the kernel space. Applications rely on system calls to gain access to critical resources, including networking, file I/O, and memory management. However, this indispensable interface simultaneously represents a substantial attack surface [7], [40], [59]. Prominent kernel vulnerabilities, such as Dirty COW (CVE-2016-5195) [10], which leverages `madvise` and the io_uring flaw (CVE-2022-0185) [12], which abuses `io_uring`, demonstrate how malicious use of system calls can lead to privilege escalation and data leakage.

Early defense mechanisms sought to detect intrusions by identifying anomalous patterns in system call activity [19], [28]. These systems monitor the order, frequency, and arguments of issued system calls, triggering alerts upon detecting deviations from established behavioral baselines. These defense mechanisms are known to be vulnerable to mimicry attacks [36], [45], [55], which evade detection by modifying system call sequences that closely resemble legitimate patterns. Such attacks deceive detectors by inserting no-op (no operation) calls, reordering system call sequences, and adjusting argument values to conform to expected behavioral profiles.

With the introduction of BPF and seccomp [39], a proactive defense research emerged that focuses on reducing the attack surface [37], an approach now widely known as *system call filtering*. This enables system call level enforcement through allowlists or denylists, blocking potentially hazardous system calls from userspace. These mechanisms are widely used in constrained environments, such as containers, sandboxes, and IoT devices, promoting the Principle of Least Privilege [8], [33], [47].

Despite their effectiveness, system call filters suffer from inherent structural limitations rooted in the modular architecture of the Linux kernel. Because filtering decisions are made based solely on system call identifiers, semantically similar calls can be used to bypass the filter. To mitigate such risks, previous work [5], [24], [46] proposed grouping *equivalent system calls*, such as `accept` and `accept4`, or `chmod` and `fchmodat`, and applying consistent filtering across these groups.

In this study, we conduct an in-depth analysis of internal kernel execution paths to uncover structural relationships among system calls that are not apparent from manual system call grouping. We observe that many system calls, though distinct in their names, invoke overlapping sets of kernel functions, including vulnerable code paths. Building on this observation, we propose a novel attack technique that leverages such internal redundancy to bypass system call filters. We define a *MimicCall* as a permitted system call that, despite differing in name or apparent function, shares internal execution pathways with a blocked call, thereby enabling access to the same vulnerable kernel functions. Through our analysis, we demonstrate that seemingly unrelated system calls can converge on common kernel routines, facilitating stealthy exploitation that evades system call filters.

In addition, we develop a dynamic analysis tool that sys-

tematically identifies MimicCalls by tracing diverse system calls and argument combinations capable of invoking a given vulnerable kernel function. Utilizing the Syzkaller project's system call descriptions, our tool generates diverse test programs with various argument structures. These programs execute while employing ftrace [49] to monitor the invoked kernel functions. The collected data are normalized and indexed, resulting in a comprehensive mapping between system calls and kernel functions, customized to the analyzed kernel environment.

We evaluated our approach in three dimensions: kernel-level execution redundancy, real-world vulnerability exposure, and filter robustness. Initially, we traced 321 system calls on a Linux kernel, computing pairwise Jaccard index for their invoked kernel functions, revealing substantial internal logic overlap even among semantically unrelated system calls. Next, we analyzed 8,927 CVE-patched kernel functions, demonstrating significant cross-invocation via multiple system calls, thereby validating the prevalence of MimicCall scenarios. Furthermore, we assessed filters generated by five representative system call filtering tools: Confine [23], C2C [25], Sysfilter [14], SysPart [46], and TSP [24], finding that they all expose vulnerable functions through MimicCall.

To illustrate MimicCall's practical applicability, we conducted case studies on known CVEs. For instance, in CVE-2016-4486, we confirmed that both `sendto` and `sendmsg` invoke the same vulnerable kernel function, allowing successful exploitation even when `sendto` was blocked. Similarly, in CVE-2016-9793, substituting `socketpair` with `socket` enabled vulnerability exploitation via permitted system call paths. In CVE-2017-7184, replacing `sendmsg` with `writev` likewise enabled successful exploitation through an alternative, permitted system call path.

These findings underscore system call filtering's intrinsic limitation: an inability to consider the kernel's internal execution semantics. Filtering policies confined solely to system call interfaces inherently remain vulnerable to evasion through alternative invocation paths that share vulnerable internal logic.

The source code for our tool is publicly available at https://github.com/koreacsl/MimicCall.

Contributions of this paper. Our key contributions include:

- Propose a novel attack technique, *MimicCalls*, that circumvents system call filters to exploit kernel vulnerabilities previously blocked.
- Design and implement an automated framework to identify *MimicCalls* via dynamic tracing of kernel-internal functions.
- Analysis leveraging real-world CVEs to validate the effectiveness of *MimicCalls* and reveal systemic gaps in current system call filter designs.

Outline. The remainder of this paper is organized as follows. Section 2 presents background on system call filtering mechanisms. Section 3 introduces our attack methodology and details the design and implementation of our analysis framework, including test case generation and dynamic tracing. Section 4 presents a comprehensive evaluation that quantifies function-level redundancy across system calls, analyzes vulnerable function exposure, and assesses the effectiveness of existing filtering tools. Section 5 demonstrates the real-world impact of our findings through case studies on known CVEs. Section 6 discusses broader implications, limitations, and directions for future research. Section 7 reviews related work on system call specialization and mimicry attacks. Section 8 concludes the paper.

2. Background

2.1. System call filtering

System call filtering is a proactive security mechanism that limits application access to kernel functionality by restricting which system calls may be invoked during execution. This is typically enforced through runtime filters that mediate system call invocations, thereby minimizing the exposed kernel surface. A widely adopted implementation of such runtime filtering is Seccomp-BPF [39], a kernel-level framework that enforces system call level access control by evaluating BPF programs against system call numbers. Its lightweight design and native integration into the Linux kernel have made it a standard foundation for deploying system call filters in production environments. Building upon this foundation, recent work [15], [20], [31] has explored the use of eBPF [16] to support more flexible and adaptive enforcement models. For example, SysXCHG [20] enables dynamic replacement of system call filters at execve time, allowing each program to enforce only the system calls it requires, rather than inheriting a monolithic policy from its parent process. Such techniques have gained traction in real-world container environments. Platforms like Kubernetes and Red Hat OpenShift apply Seccomp profiles by default to enforce system call restrictions and enhance container isolation [33], [47], while Cilium [8], a widely adopted eBPF-based security platform, integrates with Seccomp to provide fine-grained system call control in conjunction with network-layer policies.

While these deployments demonstrate the practical value of system call filtering, their effectiveness critically depends on identifying the minimal set of system calls required for correct application behavior. Over-approximation may permit unnecessary calls and weaken security, while under-approximation can disrupt functionality. A key challenge lies in accurately identifying this minimal set while avoiding both extremes. To this end, various analysis techniques have been explored.

2.2. Analysis techniques for identifying system calls

System call policies can be derived through dynamic analysis by recording the calls an application makes during runtime. DockerSlim [52] traces system calls within running containers to generate minimal Seccomp profiles, while Optimus [58] leverages eBPF tracing and statistical co-occurrence analysis to refine system call allowlists.

To minimize runtime overhead and broaden applicability, many filtering systems adopt static analysis to infer the set of reachable system calls. Sysfilter [14] uses static binary analysis and function call graph construction to estimate reachable system calls. Other systems, such as Confine [23], Chestnut [5], TAILOR [56], and C2C [25] also rely on static analysis as a foundation, but incorporate dynamic analysis to complement its limitations and refine system call policies. For instance, Confine [23] dynamically profiles the application to identify reachable libraries, and then statically analyzes those libraries and binaries using a control flow graph to extract invoked system calls. Chestnut [5] performs static analysis using compiler-based instrumentation for source code or symbolic backward execution for binaries, and optionally refines system call allowlists with runtime tracing. TAILOR [56] statically maps library functions to system calls via source-level analysis and minimally augments it with runtime profiling for environment setup. C2C [25] performs static analysis based on active configurations to identify unnecessary code paths, and applies fine-grained system call filtering using runtime data when needed.

Beyond dynamic and static profiling, several systems [24], [37], [46], [57] employ *Temporal filtering* techniques that adapt policies across different execution phases. SPEAKER [37], TSP [24] and SysPart [46] partition the program lifecycle into initialization and service phases(booting and running phase in SPEAKER [37]). These systems identify system calls used exclusively during startup, such as those involved in file loading or environment setup, and apply more restrictive filters once the application enters the service phase. Xing et al. [57] extend this concept to containerized environments by dividing execution into boot, run, and shutdown phases. They employ a hybrid analysis to determine the required system calls for each phase and enforce phase-specific filters accordingly.

Several studies [3], [38] have explored system call filtering for web platforms, notably targeting PHP, which is widely used but poses security challenges due to its dynamic features such as `eval`, `include`, and indirect calls. Saphire [3] builds a system call profile per HTTP request, while Lei et al. [38] use a finer-grained model that tracks execution states within scripts.

3. MimicCall identification

3.1. Methodology

Modern kernels use a modular design in which multiple system calls invoke shared internal core routines to improve code reuse and maintainability. Although this modularity substantially enhances software quality, it also introduces linkages between seemingly independent system calls, thereby widening the potential attack surface of system call filtering. Existing filtering techniques typically restrict access based only on system call numbers and sometimes their arguments, offering limited visibility into how execution unfolds inside the kernel once a call is invoked. As a result, a filter may unintentionally permit the execution of sensitive kernel routines through alternative call paths, allowing hidden attack vectors to persist across multiple entry points.

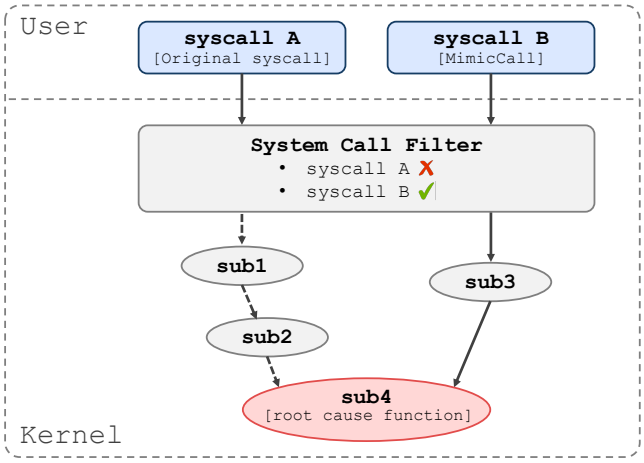

Figure 1. An Overview of the attack methodology.

In this section, we present an attack methodology that leverages this shared internal kernel structure to bypass system call filtering mechanisms. Specifically, we define the concept of a *MimicCall* as an allowed system call whose internal execution pathways overlap with those of blocked calls, thus granting indirect access to the same vulnerable kernel routines, *root cause functions*. In other words, even though a MimicCall may differ from a blocked system call in terms of its name or primary function, the shared kernel execution paths it utilizes can facilitate exploitation of otherwise inaccessible vulnerabilities.

To illustrate this attack scenario concretely, consider a root cause function, `sub4()`, originally accessible via `syscall A`, as shown in Fig. 1. Suppose a security policy explicitly denies access to `syscall A` to prevent exploitation. However, if `sub4()` is also reachable via a different system call, `syscall B`, then an attacker can bypass the filtering mechanism and exploit the vulnerability by invoking `syscall B` to trigger the same vulnerability. In this scenario, `syscall B` exemplifies a MimicCall.

Identifying MimicCalls, however, poses substantial challenges. While static analysis can in principle trace kernel call paths, real-world kernels make extensive use of indirect mechanisms such as function pointers, callbacks, and wrapper functions, and complex control flow with conditional branches further constrains static techniques. To address these limitations, we employ an automated dynamic approach that generates and executes test cases while tracing in-kernel execution to uncover MimicCalls. This method identifies permitted system-call pathways that may reach and exploit vulnerable root-cause functions. However, dynamic tracing may still miss rare or input-dependent paths, as discussed in Section 6.

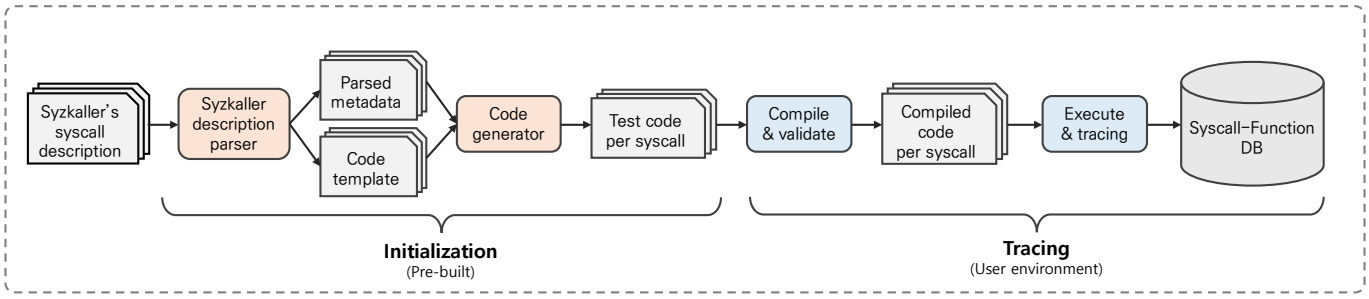

Figure 2. An overview of our tool.

3.2. Tool implementation

Overview. The primary goal of our tool is to identify permitted system calls that can invoke a given root cause function, thereby identifying potential paths to bypass system call filtering policies. To achieve this, the tool performs automated dynamic analysis and constructs a database that maps system calls to the kernel functions they reach. Given a root cause function, the tool queries this database to retrieve all allowed system calls capable of invoking it. Fig. 2 illustrates the overall architecture of our system, which operates in three distinct phases: initialization, tracing, and serving. During the initialization phase, the tool parses Syzkaller's system call definitions to extract metadata and generates minimal executable templates along with a set of valid test cases reflecting diverse argument configurations. In the tracing phase, these test cases are compiled and executed, and ftrace [49] is employed to record the kernel function call paths they invoke. To ensure the reliability of the analysis, only test cases that terminate successfully are retained for tracing. Finally, in the serving phase, the collected traces are aggregated into a reusable system call-to-function mapping database, enabling efficient post-analysis queries.

3.2.1. Initialization phase. This phase constructs the foundational dataset required for system call analysis by parsing system call definitions and automatically generating executable test programs.

Step 1: Parsing syzkaller definitions. The system begins by statically analyzing the system call definition files provided by Syzkaller. Using Syzkaller's system call definitions, we generated and executed valid test cases for 321 out of the 401 system calls supported by the framework. The remaining system calls were excluded due to stability issues or environment-specific constraints, as detailed in Appendix B. This analysis extracts key metadata, including the system call name, number and types of arguments, associated constants, and enumerated flag values. Each argument is normalized concerning its type (e.g., pointer, integer), qualifiers (e.g., const), and internal structure, if present. The parsed metadata is serialized to serve as reusable input for subsequent code generation steps.

Step 2: Generating code templates. For each system call, the system generates a minimal C code template that satisfies its basic preconditions for successful execution. This includes preparing necessary resources such as file descriptors, memory mappings, or socket connections, depending on the semantics of the call. Pointer and structure-type arguments are initialized with context-appropriate values, and required headers (e.g., `<sys/socket.h>`, `<linux/netlink.h>`) are included dynamically based on usage context. For system calls that depend on the previous state, such as accept, sendmsg, or read, the tool incorporates system call chaining to ensure that prerequisite calls (e.g., socket, connect, or open) are issued beforehand.

Step 3: Generating valid test cases. Based on the extracted metadata, the tool systematically generates valid test cases that explore different argument combinations while ensuring successful execution. Rather than aiming to trigger faults, these cases are designed to exercise a broad set of internal kernel execution paths under normal conditions. Each flag-type argument is tested in isolation by generating one test case per flag value. Integer-type arguments are set to representative boundary values within valid ranges. Constants are initialized using predefined symbolic values as specified in the system call interface. Structure-type arguments are fully populated with valid content, and pointer-type arguments are assigned either valid user-space addresses or NULL, with memory allocated accordingly. To reduce runtime overhead, each test case minimizes resource usage and explicitly releases resources after use. Generated files are systematically named to reflect the system call and argument configuration (e.g., inotify$add_watch_in_moved_to.c, fstat$at_empty_path.c), enabling organized execution and trace collection.

3.2.2. Tracing phase. This phase involves tracing the internal kernel functions invoked by the system calls generated during the execution of test cases.

Step 4: Execution and tracing. The tool compiles and executes each generated test program in a controlled environment. During compilation, necessary libraries are dynamically linked based on the requirements of each system call. For instance, `librt` is linked for system calls involving real-time timers and message queues, such as `timer` and `mq_notify`, while `libnuma` is used for memory policies and NUMA-related operations, such as `mbind`. Only binaries that execute successfully in the target environment are retained for tracing. For each valid execution, `ftrace` [49] is activated in `function_graph` mode to capture the

full sequence of kernel function calls, including call depth and hierarchical relationships, triggered during system call execution. To minimize noise and improve the accuracy of syscall-to-function mapping, tracing is restricted to the process ID of the test program, ensuring that only functions invoked by the target process are recorded. These traces enable precise mapping between the system call and the internal kernel functions it traverses. Since the test cases are designed to complete without errors, the collected traces reflect expected kernel behavior under normal conditions. To account for differences in user privilege levels, system calls that require root access are executed separately, and their traces are stored in distinct output files. This separation helps isolate privilege-dependent behavior and allows the tool to remain effective across environments where elevated permissions may not be available.

Step 5: Constructing the syscall-to-function mapping. Using the collected traces, the system builds a syscall-to-kernel function mapping database. Each entry maps a kernel function to the set of system calls that can reach it, revealing shared execution paths across system call interfaces. Since this mapping is environment-specific, the database only needs to be constructed once per kernel version and configuration. Afterward, it can be reused for multiple analyses without re-execution, improving efficiency and consistency across experiments.

3.2.3. Serving phase. Using the collected traces, our system builds a database that maps each kernel function to the set of system calls capable of reaching it, exposing shared execution paths across different system call interfaces. Since this mapping depends on the specific kernel version and configuration, it only needs to be generated once and can be reused across multiple analyses. The tool allows security analysts to identify system calls that reach a given kernel function, enabling assessment of whether a vulnerable function remains accessible despite syscall filters. This helps uncover alternative attack paths and supports tasks like policy verification and proof-of-concept development.

4. Evaluation

The goal of our evaluation is to empirically uncover how extensively internal kernel functions are shared across system calls, and to demonstrate how such implicit sharing can expose previously unrecognized attack vectors. Specifically, we aim to answer the following research questions:

- RQ1: To what extent are kernel functions shared across different system calls?
- RQ2: To what extent are root cause functions shared across different system calls?
- RQ3: How vulnerable are existing system call filtering tools to MimicCall?

To address these questions, we conduct a three-stage analysis. First, we quantify the extent to which system calls share internal kernel functions, revealing significant overlap

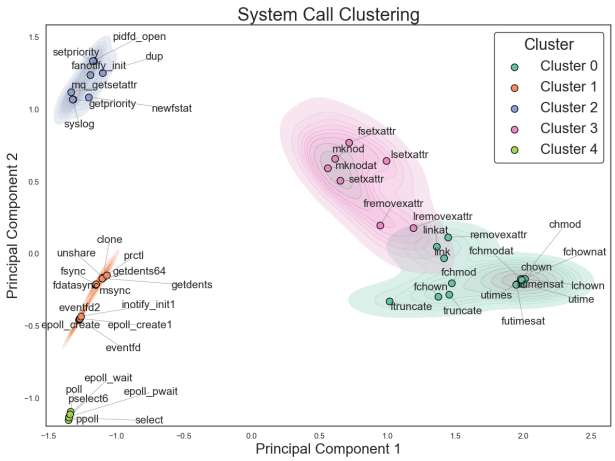

Figure 3. System call cluster

in their execution paths. Next, we examine whether this overlap includes security-critical functions by analyzing the reachability of root cause functions extracted from real-world CVEs. This reveals that multiple system calls can invoke the same root cause function, a scenario we refer to as MimicCall. Finally, we assess the extent to which these vulnerable functions remain reachable under existing filtering tools, showing that shared kernel execution paths can significantly reduce the effectiveness of system call filtering mechanisms.

Evaluation setup. Our experiments were primarily conducted on a machine equipped with an AMD Ryzen 7 5800X3D processor (8 cores, 16 threads) and 32GB of RAM. We further confirmed that the experiments remain feasible in a virtualized environment (VMware) with only 4 CPU cores and 8GB of RAM. To evaluate the applicability of our approach across diverse kernel configurations, we selected three commonly used LTS Linux kernel versions—4.4.0 (Ubuntu 16.04), 5.4.0 (Ubuntu 18.04), and 5.15.0 (Ubuntu 20.04). To enable kernel-level tracing, we ensured the `CONFIG_KALLSYMS` option remained enabled, which is the default setting in Ubuntu kernel builds.

Vulnerability dataset. To assess the security impact, we leveraged a dataset of 4,458 real-world CVEs curated by KernJC [51], which constitutes the largest publicly available collection of Linux kernel vulnerabilities. From the patch commits associated with each CVE, we extracted 8,927 unique root cause functions. We also identified which kernel versions are affected by each CVE, ensuring accuracy in our evaluation. For CVEs lacking version metadata in the KernJC dataset, we manually determined the earliest affected version by inspecting the mainline Linux kernel commit history.

4.1. Kernel function redundancy

To understand the structural commonality among system calls, we analyze their kernel-level redundancy using pairwise Jaccard index. We leverage syscall-to-function

Figure 4. Jaccard heatmaps for cluster 3 and cluster 4

mappings generated by our tracing tool, which records the internal execution paths triggered by each system call. We define the redundancy between two system calls as the size of the intersection divided by the union of the internal kernel functions they invoke.

Formally, let us assume each system call triggers a subset of internal kernel functions drawn from a global universe $\mathcal{F}$. Then, the redundancy between two system calls A and B can be defined using the Jaccard index as follows:

$$s(A, B) = \frac{|F_A \cap F_B|}{|F_A \cup F_B|}, \tag{1}$$

where $F_A \subseteq \mathcal{F}$ and $F_B \subseteq \mathcal{F}$ denote the sets of kernel functions invoked by system calls A and B, respectively. In order to isolate meaningful redundancy patterns and avoid noise from marginal overlaps, we limit our analysis to system call pairs exhibiting a Jaccard index of at least 0.7.

To further understand the shared behavior of system calls, we first analyze their coarse-grained clustering patterns. We apply Principal Component Analysis (PCA) [32] for dimensionality reduction, followed by K-Means clustering [42] with K=5, selected based on Silhouette score [50] analysis.

Fig. 3 shows the clustering result, where semantically unrelated system calls form tight clusters due to shared internal logic. Cluster 0 targets file-metadata updates: calls such as `chmod` and `truncate` share VFS routines that adjust inode attributes and link counts. Cluster 1 covers task and object management, `clone` and `fsync`, for example, create or reconfigure kernel objects while synchronising in-kernel state via scheduler, namespace, and buffer-flush paths. Cluster 4 encompasses I/O-multiplexing primitives such as `poll` and `epoll_wait`, which rely on the common poll-table, wait-queue, and wake-up logic that underpins event-driven descriptor monitoring.

To enable a finer-grained analysis of intra-cluster, we examine the pairwise overlap among system calls within each cluster using heatmaps (Fig. 4). In cluster 3 includes system calls such as `fsetxattr`, `lsetxattr`, and `setxattr`, all of which are involved in extended attribute manipulation, alongside `mknod` and `mknodat`, which handle device node creation. This cluster is thus characterized by strong reuse of kernel routines related to metadata modification and special file initialization. In contrast, Cluster 4 comprises event-driven system calls such as `poll`, `ppoll`, `select`, and `pselect6`, as well as `epoll_wait` and `epoll_pwait`, all of which share core kernel functions for polling file descriptors and managing readiness notifications.

Through this analysis, we observed that substantial system calls exhibit redundancy in their internal execution paths. This redundancy allows for the effective clustering of system calls based on shared internal routines, indicating that calls with related semantics tend to be grouped according to structural similarity in their execution. This shows the applicability of our approach to evaluating functional relationships between system calls in terms of their internal behavior. The security implications of this redundancy, particularly its effect on filtering mechanisms, are discussed in the next section.

4.2. Root cause function redundancy

To assess whether the prevalent sharing of kernel functions among system calls also implies a security weakness, we narrow our analysis to a more security-relevant subset, root cause functions extracted from real-world CVEs. As in the previous section, we compute the pairwise Jaccard index between system calls, this time based only on the root cause functions they invoke. We then cluster system calls that exhibit significant redundancy using the same PCA and K-Means-based methodology. To further understand the implications of kernel-level redundancy from a security perspective, we examined specific pairs of system calls exhibiting significant overlap in their invocation of vulnerable root-cause functions, as illustrated in Fig.5. Our analysis reveals that such redundancy is not limited to semantically similar calls; indeed, calls conventionally considered distinct or unrelated often converge on shared vulnerable kernel paths.

For instance, `chmod`, which performs permission updates, and `utime`, which performs timestamp changes, both traverse `ext4_setattr`, `__ext4_get_inode_loc`, and `inode_io_list_move_locked`. These routines have reported flaws (CVE-2015-8839, CVE-2018-10882, CVE-2024-0562) that expose a common ext4-metadata attack surface. `openat` and the extended-attribute family `*xattr` intersect at `__ext4_get_inode_loc` and `jbd2_journal_dirty_metadata` (CVE-2018-10882, CVE-2018-10883), indicating that even an `open` can exercise integrity-critical journaling paths. `fsopen` and `memfd_create` converge within memory management through `handle_mm_fault`, `drain_obj_stock`, and `kfree` (CVE-2023-3269, CVE-2021-47011, CVE-2024-36890), showing that distinct high-level intents can suffer identical memory-safety violations due to shared resource-management code.

Table 1 summarizes the number of root cause functions tracked for each kernel version, as well as the subset accessible through multiple distinct system calls (potential MimicCall candidates). For kernel 5.4.0, about 13.3% (159 out of 1,192) of all root cause functions were traceable, among which approximately 47.8% (76 out of 159) were

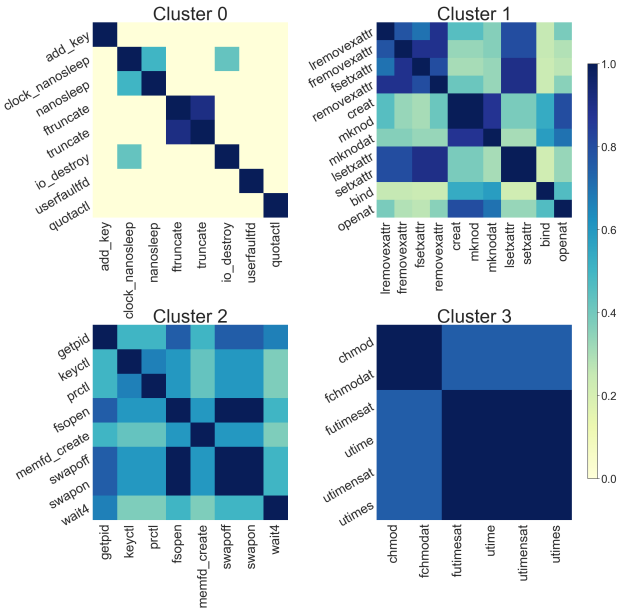

Figure 5. Jaccard index heatmaps for *MimicCall* clusters

reachable via multiple system calls. Similarly, for kernel 5.15.0, around 9.3% (108 out of 1,166) were tracked, and approximately 56.5% (61 out of 108) of these functions could be invoked by multiple system calls. This indicates significant kernel-level redundancy, highlighting potential limitations of current syscall filtering techniques.

However, an important finding from this analysis is that, beyond system calls with similar semantics, even semantically distinct system calls often access the same vulnerable kernel functions. This creates what we term MimicCall, a bypass attack scenario where an attacker can reach the same vulnerable kernel path through a different system call than the one directly associated with the vulnerability. This reveals a significant gap in current system call filtering mechanisms, as they may fail to detect these alternative access points. The next section will evaluate the impact of these vulnerabilities on filtering tools and analyze their ability to block MimicCall scenarios.

TABLE 1. ROOT CAUSE FUNCTIONS TRACKED BY OUR TOOL AND THOSE INVOKED BY MULTIPLE SYSTEM CALLS (MIMICCALL CANDIDATES)

Kernel	# of Root Cause Functions		Total
	Tracked	Multiple syscalls	
4.4.0	252 (93)	108 (35)	1,129
5.4.0	159 (59)	76 (22)	1,192
5.15.0	108 (41)	61 (23)	1,166

in parentheses indicate root privileges.

4.3. Exploiting system call filters via MimicCall

To evaluate the effectiveness of existing system call filtering tools, we examined whether the filters block all system calls that can reach the root cause functions identified in the previous section. We evaluated how many 159 root cause functions in Linux 5.4.0 remained accessible under each tool's filter.

Among the tools introduced in Section 2, our evaluation focused on those with publicly accessible implementations, as summarized in Table 2. Since each tool generates filtering policies by extracting system calls from application execution, we reused the same binaries used in their original evaluations to ensure consistency. For instance, Confine generated filters for the 148 most downloaded Docker Hub images, while C2C was evaluated on 11 widely used binaries, including curl, nginx, and wget. These two tools were evaluated using filter sets made available by the original authors through their project repositories [21], [22]. SysPart and TSP were tested on six common server binaries, such as bind, httpd, memcached, and nginx. Since the system call set in SysPart and TSP can change depending on the runtime phase, we used the filters collected during the serving phase, where the filter is more restricted. Due to its requirement for unstripped binaries, we could not reproduce the original binaries for Sysfilter and instead used the same set of binaries as SysPart and TSP for consistency.

The results reveal that all tools leave a significant portion of the attack surface exposed. Confine permitted access to 120 out of 159 vulnerable functions in the worst case, highlighting the limitations of its filtering mechanism. Other tools, such as C2C, Sysfilter, SysPart, and TSP, exhibit slightly narrower exposure ranges but consistently permit access to more than 80 root cause functions per filter.

These findings reveal a core limitation common to all evaluated system call filtering tools. While they correctly block certain system calls based on predefined policies, they overlook the internal reuse of kernel functions that allows alternative system calls to reach the same vulnerable code paths. As a result, a substantial portion of attack-relevant functionality remains accessible through allowed interfaces, reducing the practical effectiveness of these filters. This indicates that system call level access control, when applied in isolation, lacks the necessary precision to enforce meaningful attack surface reduction in modern kernels.

5. Case Study

To validate the practical impact of MimicCalls, we conduct exploit reproduction experiments in a real system environment. Specifically, for each case, we identify a MimicCall that reaches the same root cause function as the original system call, and reconstruct the attack by modifying publicly available PoCs accordingly. Among the evaluated cases, we present three PoCs that exemplify how known CVEs can be triggered through alternative paths. We then analyze how these MimicCall substitutions retrigger the original vulnerabilities and assess their ability to bypass system call filters generated by existing tools. Test environments are configured to trigger all considered PoCs, based on Ubuntu 16.04 with Linux kernel version 4.4.0 and unmodified kernel sources.

TABLE 2. EVALUATION OF SYSTEM CALL FILTERING TOOLS BASED ON THEIR EFFECTIVENESS IN BLOCKING ROOT CAUSE FUNCTIONS. FOR EACH TOOL, WE REPORT THE NUMBER OF EVALUATED FILTERS, ASSOCIATED TARGET BINARIES, AND THE RANGE (MIN/MAX) OF ACCESSIBLE ROOT CAUSE FUNCTIONS PER FILTER. A TOTAL OF 159 ROOT CAUSE FUNCTIONS ARE CONSIDERED.

Tool	# of Filters	Applications Used for Filter	# of Accessible Root Cause Functions per Filter (Min / Max of 159)
Confine	148	oracle, mysql, couchbase, postgres, busybox, ...	83 (in nats-streaming) / 120 (in nuxeo)
C2C	11	curl, httpd.bitnami, httpd.drupal, memcached, ...	101 (in smtpd) / 109 (in curl)
Sysfilter	6	bind, httpd, lighttpd, memcached, nginx, redis	100 (in httpd) / 103 (in redis)
SysPart	6	bind, httpd, lighttpd, memcached, nginx, redis	93 (in memcached) / 102 (in redis)
TSP	6	bind, httpd, lighttpd, memcached, nginx, redis	93 (in memcached) / 102 (in redis)

TABLE 3. EXPLOIT REPRODUCTION VIA MIMICCALL ON REAL-WORLD KERNEL VULNERABILITIES

CVE	Exploit Type (CVSS)	Original Syscall	MimicCall	Bypassed Filters
CVE-2016-4486	Information Leak (3.3 LOW)	`sendto`	`sendmsg`	Confine (6/148), C2C (1/11)
CVE-2016-9793	Privilege Escalation / DoS (7.8 HIGH)	`socketpair`	`socket`	Confine (59/148), C2C (7/11) Sysfilter (3/6), SysPart (4/6), TSP (4/6)
CVE-2017-7184	Privilege Escalation / DoS (7.8 HIGH)	`sendmsg`	`writev`	C2C (4/11), Sysfilter (2/6), SysPart (3/6), TSP (4/6)

5.1. CVE-2016-4486

CVE-2016-4486 [9] is a kernel information disclosure vulnerability introduced by the insufficient initialization of a kernel data structure. Specifically, the `rtnl_fill_link_ifmap()` fails to fully initialize a struct `rtnl_link_ifmap` instance when handling a Netlink request. As a result, uninitialized stack memory can be leaked to user space via the `IFLA_MAP` attribute of the Netlink response.

The original PoC [17] exploits this behavior using the `sendmsg` system call to transmit a crafted Netlink message and extract leaked stack bytes from the kernel's response.

```
if (send(rtnetlink_sk, &req, req.nh.nlmsg_len,
    0) < 0) {
    printf("send error\n");
    goto out;
}
```

Listing 1. Original PoC using send()

Our analysis shows that `sendmsg` acts as a MimicCall in this scenario by reaching the same root cause function, `rtnl_fill_link_ifmap()`, originally triggered by `sendto`. We modified the original PoC to use `sendmsg` with properly configured structures and confirmed that it successfully triggered the vulnerability and leaked uninitialized stack memory.

```
struct msghdr msg;
struct iovec iov;
iov.iov_base = &req;
iov.iov_len = req.nh.nlmsg_len;
msg.msg_name = NULL;
msg.msg_namelen = 0;
msg.msg_iov = &iov;
msg.msg_iovlen = 1;
msg.msg_control = NULL;
msg.msg_controllen = 0;
msg.msg_flags = 0;

if (sendmsg(rtnetlink_sk, &msg, 0) < 0) {
    printf("sendmsg error\n");
    goto out;
}
```

Listing 2. Modified PoC using sendmsg()

To evaluate the feasibility of a MimicCall-based attack, we examined system call filtering results produced by multiple tools. Among them, we specifically searched for profiles in which the original system call, `sendto`, was blocked while the MimicCall, `sendmsg`, remained permitted. We identified such a case in the system call filter generated by Confine for several application binaries, including django, haskell, python, rails, ruby, and swipl. Under these profiles, the original exploit would be blocked under such policies, but the modified PoC using MimicCall could still reach the vulnerable code path.

5.2. CVE-2016-9793

CVE-2016-9793 is a privilege escalation vulnerability in the Linux kernel's `sock_setsockopt()`, where improper handling of negative values for the `sk_sndbuf` and `sk_rcvbuf` fields can lead to integer overflows. When exploited, this flaw allows memory corruption and, under certain conditions, arbitrary code execution. The vulnerability is triggered via the `SO_SNDBUFFORCE` or `SO_RCVBUFFORCE` options, which permit overriding standard buffer limits. This allows a local attacker with the

CAP_NET_ADMIN capability to influence how the kernel interprets memory references.

The original PoC constructs a socket pair using specific arguments and invokes setsockopt to assign a negative buffer size, resulting in the kernel referencing a user-mapped skb_shared_info structure. By setting the destructor_arg field to point to a user-defined callback, the exploit gains root privileges when the socket is later freed.

```
rv = socketpair(AF_LOCAL, SOCK_STREAM, 0, &
    sockets[0]);
```

Listing 3. Original PoC using socketpair()

To evaluate the feasibility of a MimicCall-based variant, we replaced socketpair with the more generic socket system call and manually established a bidirectional socket connection using the AF_UNIX domain. Unlike socketpair, which is a higher-level interface often used for inter-process communication and therefore more likely to be restricted by syscall filters, socket is broadly permitted across application profiles.

```
const char *SOCK_PATH = "/tmp/mimic";
struct sockaddr_un addr_un;
unlink(SOCK_PATH);
int listener = socket(AF_UNIX, SOCK_STREAM, 0);
memset(&addr_un, 0, sizeof(struct sockaddr_un));
addr_un.sun_family = AF_UNIX;
strncpy(addr_un.sun_path, SOCK_PATH, sizeof(
    addr_un.sun_path) - 1);
bind(listener, (struct sockaddr *)&addr_un,
    sizeof(struct sockaddr_un));
listen(listener, 1);
sockets[1] = socket(AF_UNIX, SOCK_STREAM, 0);
connect(sockets[1], (struct sockaddr *)&addr_un,
     sizeof(struct sockaddr_un));
sockets[0] = accept(listener, NULL, NULL);
close(listener);
```

Listing 4. Modified PoC using socket()

We further assessed the bypassability of this MimicCall variant by analyzing syscall filters generated by existing tools. Among 148 application-specific filters generated by Confine, 59 explicitly blocked socketpair, while socket remained allowed in most cases. Similarly, C2C blocked socketpair in 7 out of 11 binaries, Sysfilter in 3 out of 6, and both SysPart and TSP in 4 out of 6.

Furthermore, we demonstrated that the MimicCall framework can be extended to automatically generate alternative PoCs and exploit variants by analyzing system call usage patterns within PoCs using regular expressions and matching them against a pre-constructed database of similar system call pairs.

5.3. CVE-2017-7184

CVE-2017-7184 is a privilege escalation vulnerability in the Linux kernel that stems from insufficient bounds checking within the xfrm_replay_verify_len(). When handling a Netlink message of type XFRM_MSG_NEWAE, the kernel fails to validate the size of the xfrm_replay_state_esn structure and its accompanying bitmap buffer. This flaw can be exploited by a local attacker with the CAP_NET_ADMIN capability to access memory beyond its intended bounds.

The original PoC [18] initializes a Netlink socket with the NETLINK_XFRM protocol and uses sendmsg to inject crafted XFRM_MSG_NEWSA and XFRM_MSG_NEWAE messages. These messages include an oversized bitmap buffer attached as a XFRMA_REPLAY_ESN_VAL attribute. When processed by the kernel, the crafted bitmap causes Out-of-Bound access in the replay window handling logic, ultimately corrupting kernel memory and enabling root access through manipulated credentials.

```
msg.msg_iov = &iov;
msg.msg_iovlen = 1;
err = sendmsg (sock, &msg, 0);
return err;
...

msg.msg_namelen = sizeof(nladdr);
msg.msg_iov = &iov;
msg.msg_iovlen = 1;
err = sendmsg (sock, &msg, 0);
return err;
...

msg.msg_iovlen = 1;
err = sendmsg (sock, &msg, 0);
return err;
```

Listing 5. Original PoC using sendmsg()

Building on this observation, we implemented a PoC variant in which sendmsg is replaced with writev. The modified version first establishes a raw socket connection using connect and then transmits the crafted payload using writev.

```
connect(sock,(void *)&(nladdr),sizeof(nladdr));
err = writev(sock, &iov, 1);
return err;
...

connect(sock,(void *)&(nladdr),sizeof(nladdr));
err = writev(sock, &iov, 1);
return err;
...

connect(sock,(void *)&(sai),sizeof(sai));
err = writev(sock, &iov, 1);
return err;
```

Listing 6. Modified PoC using writev()

Furthermore, we evaluated the effectiveness of syscall filters generated by existing tools. Notably, the filters produced by SysPart for applications such as httpd, lighttpd, and redis blocked sendmsg, while still allowing writev. Similarly, TSP filters for httpd, lighttpd, bind, and redis exhibited the same pattern.

6. Discussion

Coverage and misses in dynamic collection. Our mapping from system calls to internal kernel functions is obtained via dynamic tracing, which provides runtime evidence

but inevitably yields incomplete coverage. In particular, test generation is guided by interface descriptions from `Syzkaller` [29]; thus, the observed function sets depend on the accuracy and completeness of these specifications and on the execution contexts in which tests succeed. As a result, exception-handling and error paths may be underexplored, and kernel version or configuration differences can affect traceability. The overlaps we report should therefore be interpreted as a conservative lower bound rather than an exhaustive enumeration of all reachable pathways.

False positives and exploitability. Reachability of a vulnerable routine is necessary but not sufficient for exploitability. Even when two system calls traverse the same root-cause function, argument constraints and runtime context (e.g., privilege, namespace, timing) may differ enough to preclude triggering the flaw along the alternative path. For example, in our observation of CVE-2016-0728, the original route via `keyctl()` steers `join_session_keyring(name)` with attacker-influenced attributes. In contrast, the alternative via `add_key()/request_key()` resolves to `join_session_keyring(NULL)`, which effectively forces an anonymous session keyring and limits manipulation of the `cred`/keyring state. As a result, exploit reproduction was difficult along the latter path. To reduce such false positives, one practical direction is to complement tracing with additional analysis that checks the controllability of the arguments required at the root-cause function. For example, lightweight taint analysis [43] can be used to verify that attacker-supplied inputs can reach the relevant parameters or guarding predicates. We leave systematic integration and evaluation of such analyses to future work.

Dependency on kernel symbol visibility Our analysis leverages ftrace to identify kernel functions invoked by system calls, which depends on kernel symbols. As shown in Evaluation 4.2, 8,927 root cause functions were extracted from patch data, of which 14% were accessible via the kernel symbol table. This limitation may be mitigated by selectively exposing internal functions or complementing tracing with lightweight static analysis of caller–callee relationships. In this work, we focus on directly traceable functions to ensure reliable and reproducible analysis across standard kernel builds.

Mitigation strategies. Our findings suggest that system call level policies may be bypassed by exploiting MimicCalls. To address this, incorporating internal function-level policies could improve the precision of existing filtering mechanisms. When the target function is known, eBPF-based tools such as `kprobe` [54] can enable detailed inspection at the execution point of the flaw. This may be particularly useful during the period between vulnerability disclosure and patch deployment, allowing for more targeted and timely mitigation. In contrast, when the vulnerable function is unknown, as in zero-day scenarios, MimicCall can help identify system calls with high internal similarity. Security policies may then enhance protection by applying stricter runtime controls to these candidates, such as argument validation [44], [48] or anomaly-based invocation checks [30]. This approach may be especially effective in constrained environments, such as embedded systems, where the system call surface is narrow and more amenable to targeted monitoring.

Extension to sequence-based detection. Many sequence-based detection mechanisms represent system call behavior as fixed patterns. These include n-gram [6], [53], probabilistic [4], machine learning [41], and state-based models [6]. Because these approaches operate at the system call level, substitutions using MimicCalls can generate numerous variants that preserve the structure of known patterns, potentially enabling evasion. We leave a systematic empirical study of this direction to future work.

7. Related Work

7.1. System call filter bypass techniques

System call filtering has previously shown structural limitations, notably in the case of the x32 ABI, an alternative interface used on x86_64 Linux kernels. This ABI uses 32-bit pointers in a 64-bit environment and assigns system call numbers by adding a fixed offset (__X32_SYSCALL_BIT, or 0x40000000). Thus, the same logical system call may have different numbers depending on the ABI. If a seccomp policy filters only standard system call numbers, an attacker can bypass it by invoking the x32 variant. The official Linux manual [39] warns of this and recommends covering x32 system calls in filtering policies.

A different class of bypass exploits the temporal gap between filtering and execution. In CVE-2019-2054 [11], an attacker uses ptrace to intercept a traced process and modify its system call number immediately before execution. Although the original system call (e.g., gettid()) is permitted by the seccomp filter, it is replaced with a prohibited system call (e.g., swapon()) after the filter decision has been made. Because kernels before version 4.8 do not re-evaluate the system call at the execution point, the modified call executes without further validation.

A related issue, assigned CVE-2022-30594 [13], involves the misuse of PTRACE_SEIZE with the PTRACE _O_SUSPEND_SECCOMP flag. This option is intended to temporarily suspend seccomp filtering during tracing and should only be available to privileged tracers. However, due to a missing capability check in the kernel's implementation, unprivileged processes are able to attach to seccomp-constrained targets and activate this mode. Once enabled, seccomp enforcement is suspended at the kernel level, causing all subsequent system calls to bypass the filter entirely.

Whereas the above attacks rely on ABI mismatches, timing inconsistencies, or tracer-based mechanisms, the present work focuses on a different dimension of bypass. It examines how disparate system calls can share internal kernel execution paths, allowing one call to mimic the behavior of another. As a result, the examined technique remains effective under strict enforcement conditions, without requiring misconfiguration, elevated privileges, or auxiliary attack channels.

7.2. System call sequence-based anomaly detection

Detecting intrusions based on system call patterns has been a longstanding area of research since early efforts in anomaly detection [19], [28]. These approaches monitor system call sequences, frequencies, and argument patterns using various techniques and trigger alerts when deviations from predefined normal behaviors are observed. More recent work has proposed increasingly sophisticated sequence-based detection mechanisms. ReplicaWatcher [35] collects system call events from container replicas and extracts behavioral features such as call frequency, type, and inter-call latency. By quantifying behavioral dissimilarities between replicas using similarity and statistical metrics, it derives baseline patterns and identifies deviations indicative of anomalous activity. μPolicyCraft [2] performs static analysis of binaries to construct Effect graphs that capture the control and data flow of system call sequences and arguments. These graphs are then compiled into finite-state automata that encode permissible invocation paths, enabling runtime enforcement of expected behavior and blocking of malicious flows. Phoenix [34] generalizes repeated attack scenarios into abstract system call patterns and models them as sequence-aware finite-state machines. Its enforcement strategy combines coarse-grained filtering via seccomp with fine-grained argument inspection using ptrace, supporting layered and precise runtime detection. These approaches operate during execution and rely on the state to detect abnormal sequences.

In contrast, our work targets stateless pre-execution filtering mechanisms themselves, revealing that they can be circumvented. These sequence-based approaches are known to be vulnerable to mimicry attacks, and this has led to the development of detection mechanisms that have evolved to be more resilient against such evasion techniques. Related work on mimicry attacks is discussed in more detail in the following section.

7.3. Mimicry attack

Early research has demonstrated that HIDS relying on system call sequences is structurally vulnerable to mimicry attacks [36], [45], [55].

Wagner and Soto [55] introduced mimicry attacks as a method to bypass system call-based HIDS by preserving malicious intent while imitating benign behavior. Their approach involved inserting no-op system calls, modifying arguments, and reordering call sequences to construct traces that remain consistent with legitimate execution patterns. To formalize this strategy, they modeled the normal behavior of the IDS as a finite state automaton and represented the attacker's goal as a regular language. By analyzing the intersection of these two languages, they demonstrated how to generate evasive sequences within acceptable behavior.

This line of research was extended by Kruegel et al. [36], who proposed an automated mimicry attack generation technique based on static binary analysis and symbolic execution. Their system analyzes x86 binaries to identify control-flow paths that allow the attacker to temporarily divert execution into a benign context, then redirect control flow back to malicious code using manipulated return addresses or function pointers. This enables system call traces to appear legitimate, effectively evading detectors that rely on program counters or call stack context.

More recently, Goyal et al. [27] demonstrated that provenance-based HIDS (Prov-HIDS) are vulnerable to mimicry attacks as well. These systems represent execution behavior as graphs of system events, which are processed using modern graph embedding techniques. The authors showed that attackers can manipulate an attack's provenance graph to resemble benign behavior by injecting subgraphs derived from legitimate processes through three evasion strategies. These methods were shown to successfully evade Prov-HIDS in experimental settings.

Building on these insights, our approach aligns with the fundamental objective of mimicry attacks, which is to preserve malicious intent while conforming to expected normal patterns. In contrast to previous techniques that manipulate system call sequences, our method focuses on individual system calls as the unit of mimicry.

8. Conclusion

In this work, we introduced and systematized *MimicCalls*, a previously unrecognized class of system call filter bypasses that exploit the internal reuse of kernel functions across distinct interfaces. To uncover such bypasses, we developed an automated framework that integrates system call specifications from Syzkaller with dynamic tracing via ftrace, enabling the construction of kernel-specific mappings between system calls and their underlying function invocations. Applying this framework to 321 system calls revealed extensive structural overlap in their execution paths, as quantified by pairwise Jaccard index, even among calls with no semantic relation. To assess the real-world impact of this overlap, we analyzed 8,927 CVE-patched kernel functions and found that many could be reached through multiple system calls, providing empirical evidence for the prevalence of MimicCall scenarios in practical vulnerability contexts. We then evaluated five representative filtering tools, Confine, C2C, Sysfilter, SysPart, and TSP, and observed that vulnerable functions remained accessible via permitted system calls, despite the use of restrictive policies. To demonstrate the security implications, we successfully reproduced three real-world CVEs using MimicCalls that substituted for originally blocked system calls under actual filter constraints. These findings highlight a fundamental limitation in existing system call filtering: the gap between interface-level policies and internal execution logic creates hidden attack surfaces that are systematically overlooked, leaving critical kernel functionality exposed to adversarial reuse.

Acknowledgement

This research was supported by a National Research Foundation of Korea (NRF) grant, funded by the Korean government (MSIT) (RS-2023-NR077166, RS-2023-00227165). This research was supported by the Korea University Grant.

References

[1] Mojtaba Bagherzadeh, Nafiseh Kahani, Cor-Paul Bezemer, Ahmed E. Hassan, Juergen Dingel, and James R. Cordy. Analyzing a decade of Linux system calls. *Empirical Software Engineering*, 23:1519–1551, 2018.

[2] William Blair, Frederico Araujo, Teryl Taylor, and Jiyong Jang. Automated synthesis of effect graph policies for microservice-aware stateful system call specialization. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P 2024)*, pages 4554–4572, 2024.

[3] Alexander Bulekov, Rasoul Jahanshahi, and Manuel Egele. Saphire: Sandboxing PHP applications with tailored system call allowlists. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security 2021)*, pages 2881–2898, 2021.

[4] Jeffrey Byrnes, Thomas Hoang, Nihal Nitin Mehta, and Yuan Cheng. A modern implementation of system call sequence based host-based intrusion detection systems. In *Proceedings of the 2020 Second IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA 2020)*, pages 218–225. IEEE, 2020.

[5] Claudio Canella, Mario Werner, Daniel Gruss, and Michael Schwarz. Automating seccomp filter generation for Linux applications. In *Proceedings of the 2021 Cloud Computing Security Workshop (CCSW 2021)*, pages 139–151, 2021.

[6] Raymond Canzanese, Spiros Mancoridis, and Moshe Kam. Run-time classification of malicious processes using system call analysis. In *Proceedings of the 2015 10th International Conference on Malicious and Unwanted Software (MALWARE 2015)*, pages 21–28. IEEE, 2015.

[7] Yueqi Chen, Zhenpeng Lin, and Xinyu Xing. A systematic study of elastic objects in kernel exploitation. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS 2020)*, pages 1165–1184, 2020.

[8] Securing networks with cilium. https://docs.cilium.io/en/latest/reference-guides/bpf/architecture/, 2025. Accessed: 2025-05-26.

[9] CVE-2016-4486: Nvd pages. https://nvd.nist.gov/vuln/detail/CVE-2016-4486, 2025. Accessed: 2025-05-21.

[10] CVE-2016-5195: Nvd pages. https://nvd.nist.gov/vuln/detail/CVE-2016-5195, 2025. Accessed: 2025-05-21.

[11] CVE-2019-2054: ptrace + seccomp-bpf enforcement gap. https://nvd.nist.gov/vuln/detail/CVE-2019-2054, 2025. Accessed: 2025-05-21.

[12] CVE-2022-0185: Nvd pages. https://nvd.nist.gov/vuln/detail/CVE-2022-0185, 2025. Accessed: 2025-05-21.

[13] CVE-2022-30594: Missing privilege check in PTRACE_SEIZE + PTRACE_O_SUSPEND_SECCOMP. https://nvd.nist.gov/vuln/detail/CVE-2022-30594, 2025. Accessed: 2025-05-21.

[14] Nicholas DeMarinis, Kent Williams-King, Di Jin, Rodrigo Fonseca, and Vasileios P. Kemerlis. Sysfilter: Automated system call filtering for commodity software. In *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, pages 459–474, 2020.

[15] Qiqing Deng, Yanqiang Zhang, Zhen Xu, Qian Tan, and Yan Zhang. Conprov: A container-aware provenance system for attack investigation. In *Proceedings of the 2024 Annual Computer Security Applications Conference (ACSAC 2024)*, pages 89–101, 2024.

[16] eBPF – what is eBPF? https://ebpf.io/ko-kr/what-is-ebpf/, 2025. Accessed: 2025-05-21.

[17] ExploitDB. Poc code for CVE-2016-4486. https://www.exploit-db.com/exploits/46006, 2025. Accessed: 2025-05-21.

[18] ExploitDB. Poc code for CVE-2017-7184. https://github.com/snorez/exploits/blob/master/cve-2017-7184/exp.c, 2025. Accessed: 2025-05-21.

[19] Stephanie Forrest, Steven A. Hofmeyr, Anil Somayaji, and Thomas A. Longstaff. A sense of self for UNIX processes. In *Proceedings of the 1996 IEEE Symposium on Security and Privacy (S&P 1996)*, pages 120–128. IEEE, 1996.

[20] Alexander J. Gaidis, Vaggelis Atlidakis, and Vasileios P. Kemerlis. Sysxchg: Refining privilege with adaptive system call filters. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS 2023)*, pages 1964–1978, 2023.

[21] Seyedhamed Ghavamnia. Sample system call filters generated by confine. https://github.com/shamedgh/confine/tree/master/sample.results, 2020. Accessed: 2025-05-19.

[22] Seyedhamed Ghavamnia. System call filters generated by c2c. https://github.com/shamedgh/c2c/tree/main/outputs.complete, 2022. Accessed: 2025-05-19.

[23] Seyedhamed Ghavamnia, Tapti Palit, Azzedine Benameur, and Michalis Polychronakis. Confine: Automated system call policy generation for container attack surface reduction. In *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, pages 443–458, 2020.

[24] Seyedhamed Ghavamnia, Tapti Palit, Shachee Mishra, and Michalis Polychronakis. Temporal system call specialization for attack surface reduction. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security 2020)*, pages 1749–1766, 2020.

[25] Seyedhamed Ghavamnia, Tapti Palit, and Michalis Polychronakis. C2c: Fine-grained configuration-driven system call filtering. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS 2022)*, pages 1243–1257, 2022.

[26] Fernando Godínez, Dieter Hutter, and Raúl Monroy. On the use of word networks to mimicry attack detection. In *Proceedings of the 2006 International Conference on Emerging Trends in Information and Communication Security (ETRICS 2006)*, pages 423–435, 2006.

[27] Akul Goyal, Xueyuan Han, Gang Wang, and Adam Bates. Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems. In *Proceedings of the 30th Network and Distributed System Security Symposium (NDSS 2023)*, 2023.

[28] Steven A. Hofmeyr, Stephanie Forrest, and Anil Somayaji. Intrusion detection using sequences of system calls. *Journal of Computer Security*, 6(3):151–180, 1998.

[29] Google Inc. Syzkaller: Kernel fuzzer. https://github.com/google/syzkaller, 2025. Accessed: 2025-05-19.

[30] Christopher Jelesnianski, Mohannad Ismail, Yeongjin Jang, Dan Williams, and Changwoo Min. Protect the system call, protect (most of) the world with bastion. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2023)*, volume 3, pages 528–541, 2023.

[31] Jinghao Jia, YiFei Zhu, Dan Williams, Andrea Arcangeli, Claudio Canella, Hubertus Franke, Tobin Feldman-Fitzthum, Dimitrios Skarlatos, Daniel Gruss, and Tianyin Xu. Programmable system call security with eBPF. *arXiv preprint arXiv:2302.10366*, 2023.

[32] Ian T. Jolliffe. *Principal Component Analysis*. Springer, 2002.

[33] Kubernetes security – seccomp profiles. https://kubernetes.io/docs/tutorials/security/seccomp/, 2025. Accessed: 2025-05-23.

[34] Hugo Kermabon-Bobinnec, Yosr Jarraya, Lingyu Wang, Suryadipta Majumdar, and Makan Pourzandi. Phoenix: Surviving unpatched vulnerabilities via accurate and efficient filtering of syscall sequences. In *Proceedings of the 31st Network and Distributed System Security Symposium (NDSS 2024)*, 2024.

[35] Asbat El Khairi, Marco Caselli, Andreas Peter, and Andrea Continella. Replicawatcher: Training-less anomaly detection in containerized microservices. In *Proceedings of the Network and Distributed System Security Symposium (NDSS 2024)*, 2024.

[36] Christopher Kruegel, Engin Kirda, Darren Mutz, William Robertson, and Giovanni Vigna. Automating mimicry attacks using static binary analysis. In *Proceedings of the 14th USENIX Security Symposium (USENIX Security 2005)*, volume 14, pages 11–11, 2005.

[37] Lingguang Lei, Jianhua Sun, Kun Sun, Chris Shenefiel, Rui Ma, Yuewu Wang, and Qi Li. Speaker: Split-phase execution of application containers. In *Proceedings of the 14st Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA 2017)*, pages 230–251. Springer, 2017.

[38] Yunsen Lei and Craig A. Shue. Making (only) the right calls: Preventing remote code execution attacks in PHP applications with contextual, state-sensitive system call filtering. In *Proceedings of the 21st Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA 2025)*, 2025. To appear.

[39] Linux man-pages project. seccomp(2) – Linux manual page. https://man7.org/linux/man-pages/man2/seccomp.2.html, 2025. Accessed: 2025-05-21.

[40] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, et al. Meltdown: Reading kernel memory from user space. *Communications of the ACM*, 63(6):46–56, 2020.

[41] ShaoHua Lv, Jian Wang, YinQi Yang, and Jiqiang Liu. Intrusion prediction with system-call sequence-to-sequence model. *IEEE Access*, 6:71413–71421, 2018.

[42] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.

[43] James Newsome and Dawn Song. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In *Proceedings of the 12th Network and Distributed System Security Symposium (NDSS 2005)*, 2005.

[44] Shankara Pailoor, Xinyu Wang, Hovav Shacham, and Isil Dillig. Automated policy synthesis for system call sandboxing. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–26, 2020.

[45] Chetan Parampalli, R. Sekar, and Rob Johnson. A practical mimicry attack against powerful system-call monitors. In *Proceedings of the 2008 ACM Symposium on Information, Computer and Communications Security (CCS 2008)*, pages 156–167, 2008.

[46] Vidya Lakshmi Rajagopalan, Konstantinos Kleftogiorgos, Enes Göktas, Jun Xu, and Georgios Portokalidis. Syspart: Automated temporal system call filtering for binaries. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS 2023)*, pages 1979–1993, 2023.

[47] Seccomp profiles – red hat openshift documentation. https://docs.redhat.com/en/documentation/openshift_container_platform/4.14/html/security_and_compliance/seccomp-profiles, 2025. Accessed: 2025-05-23.

[48] Maryam Rostamipoor, Seyedhamed Ghavamnia, and Michalis Polychronakis. Confine: Fine-grained system call filtering for container attack surface reduction. *Computers & Security*, 132:103325, 2023.

[49] Steven Rostedt. Ftrace – function tracer. https://www.kernel.org/doc/html/latest/trace/ftrace.html, 2025. Accessed: 2025-05-26.

[50] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

[51] Bonan Ruan, Jiahao Liu, Chuqi Zhang, and Zhenkai Liang. KernJC: Automated vulnerable environment generation for Linux kernel vulnerabilities. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2024)*, pages 384–402, 2024.

[52] Slim.AI Team. DockerSlim (slimtoolkit): Minify and secure Docker containers. https://github.com/slimtoolkit/slim, 2025. Accessed: 2025-05-26.

[53] Somin Song, Sahil Suneja, Michael V. Le, and Byungchul Tak. On the value of sequence-based system call filtering for container security. In *Proceedings of the 2023 IEEE 16th International Conference on Cloud Computing (CLOUD 2023)*, pages 296–307, 2023.

[54] The Linux Kernel Organization. Kprobes documentation. https://docs.kernel.org/trace/kprobes.html, 2023. Accessed: 2025-05-27.

[55] David Wagner and Paolo Soto. Mimicry attacks on host-based intrusion detection systems. In *Proceedings of the 2002 ACM SIGSAC Conference on Computer and Communications Security (CCS 2002)*, pages 255–264, 2002.

[56] Yunlong Xing, Jiahao Cao, Kun Sun, Fei Yan, and Shengye Wan. The devil is in the detail: Generating system call whitelist for Linux seccomp. *Future Generation Computer Systems*, 135:105–113, 2022.

[57] Yunlong Xing, Xinda Wang, Sadegh Torabi, Zeyu Zhang, Lingguang Lei, and Kun Sun. A hybrid system call profiling approach for container protection. *IEEE Transactions on Dependable and Secure Computing*, 21(3):1068–1083, 2023.

[58] Seungyong Yang, Brent Byunghoon Kang, and Jaehyun Nam. Optimus: Association-based dynamic system call filtering for container attack surface reduction. *Journal of Cloud Computing*, 13(1):71, 2024.

[59] Kyle Zeng, Zhenpeng Lin, Kangjie Lu, Xinyu Xing, Ruoyu Wang, Adam Doupé, Yan Shoshitaishvili, and Tiffany Bao. Retspill: Igniting user-controlled data to burn away Linux kernel protections. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS 2023)*, pages 3093–3107, 2023.

Appendix

Appendix A.
Equivalent System Call Lists from Prior Work

Table A.1: TSP [24] and SysPart [46]

System Call	Equivalent(s)
execve	execveat
accept	accept4
dup	dup2, dup3
eventfd	eventfd2
chmod	fchmodat
recv	recvfrom, read
send	sendto, write
open	openat
select	pselect6, epoll_wait, epoll_wait_old poll, ppoll, epoll_pwait

Table A.2: Godínez et al. [26]

System Call	Equivalent(s)
memcntl	mctl
execv	execve, execvp, execl, execle, execlp
read	pread, readv, mmap
write	pwrite, writev
open_read	read
open_write	write
exit	_exit
acl	facl
chdir	fchdir
chmod	fchmod
chown	fchown, lchown
stat	fstat
brk	sbrk

Table A.3: Canella et al. [5]

System Call	Equivalent(s)
munlockall	munlock
mlockall	mlock, mlock2
execve	execveat
mknod	mknodat
recvfrom	recvmsg, recvmmsg
writev	pwritev
getdents	getdents64
sendto	sendmmsg, sendmsg
rename	renameat, rename2
accept	accept4
open	openat
epoll_ctl	epoll_ctl_old
epoll_create	epoll_create1
listxattr	llistxattr, flistxattr
getxattr	fgetxattr, lgetxattr

Table A.4: Bagherzadeh et al. [1]

Sibling Type	Examples
Parameter extension [2, 3, ...]	dup, dup2
Architecture compatibility [*32, *64]	truncate, truncate64
Working directory support [*at]	open, openat
Backwards compatibility [*old]	vm86, vm86old
Real-time variants [rt*]	sigreturn, rt_sigreturn
Miscellaneous	waitpid, wait4

Appendix B.
Excluded System Calls

Reason for Exclusion	Examples
Too recent in our environment	mseal
32-bit compatibility only [*_32]	setuid32
Obsolete legacy syscalls [old*]	oldstat
time64 variants [*_time64]	sendfile64
System-disruptive ops	kexec_load
Untraced or unused	sigprocmask